

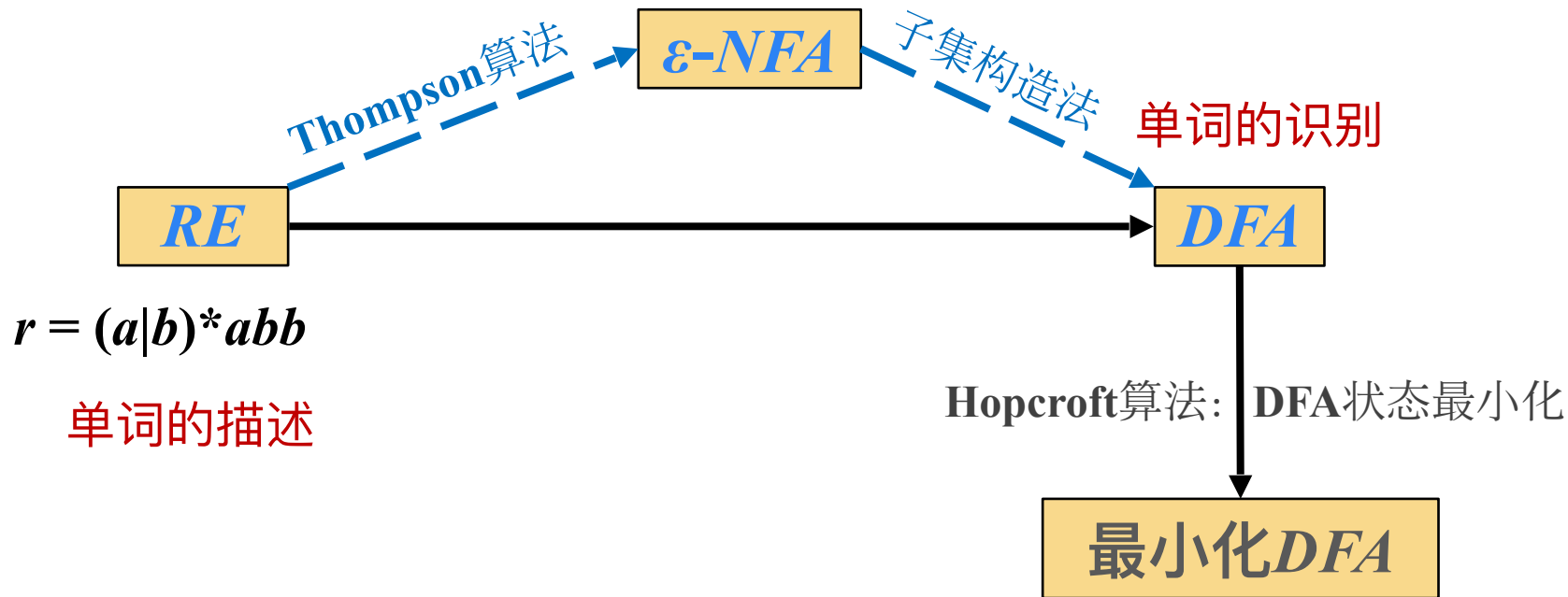


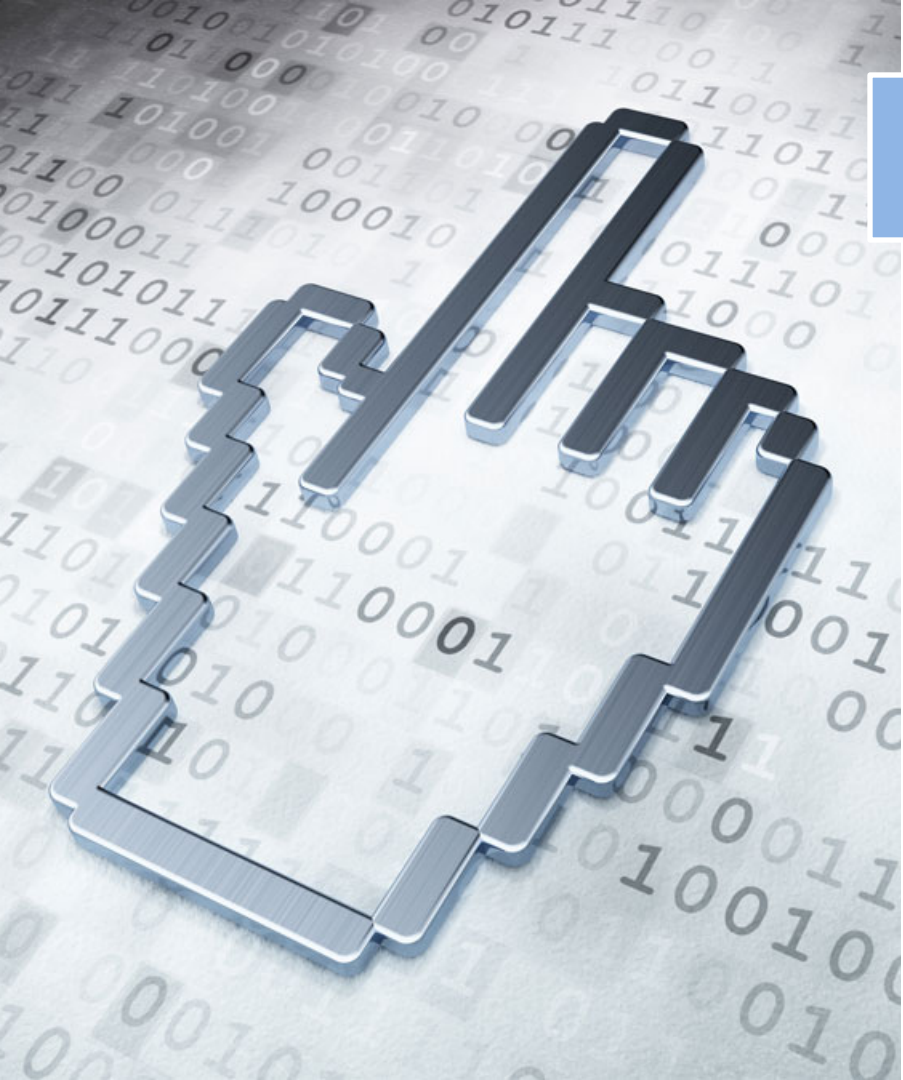
编译系统
第三章
词法分析



哈尔滨工业大学 陈鄞 单丽莉

3.2.3 从正则表达式到有穷自动机





本章内容

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

编译器的结构

position = initial + rate * 60;



<标识符, **position**>

<=**>**

<标识符, **initial**>

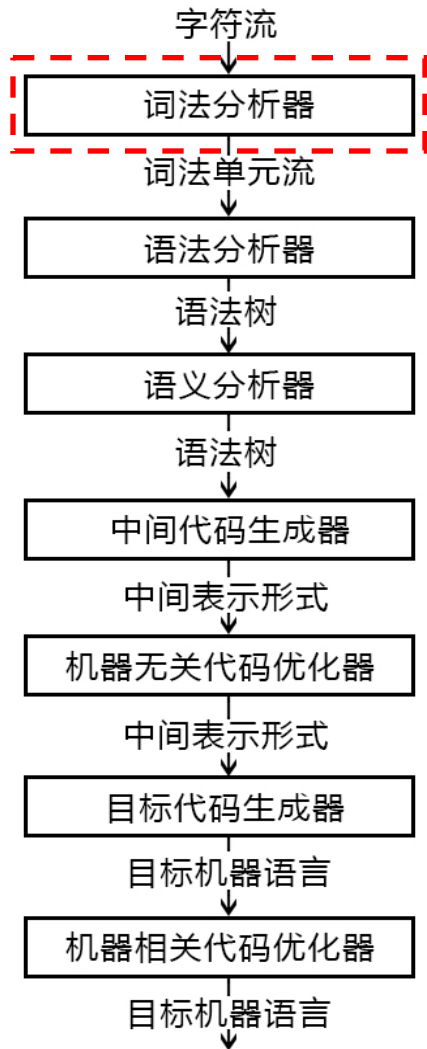
<+>

<标识符, **rate**>

<*>

<数值, **60**>

<;>



➤任务：

- 从左到右扫描源程序，根据词法规则识别单词，并转换成统一表示的单词(token)串；同时，
 - 删掉空格字符和注释；
 - 对常数完成数字字符串到数值的转换；
 - 检查词法错误，记录行号并报错；
 - 错误恢复，以便继续进行词法分析；

➤输出：词法单元序列（token序列）

- 词法单元表示：二元组 <符号名字，属性值>

词法单元符号设计

➤ 词法单元 token:

< 符号名字, 属性值 >

	单词类型	种别	符号名字	属性值(词法值)
1	关键字	program、if、else、then、...	一词一符	无
2	数据类型	int、float、char、bool ...	type	类型字符串(词素)
2	标识符	变量名、数组名、记录名、过程名、...	id	名字字符串(词素)
3	常量	整型、浮点型、字符型、布尔型、...	一类一符 INT、FLOAT...	数值
4	算术运算符	+ - * / ++ --	一词一符	无
5	关系运算符	> < == != >= <=	relop	运算符(词素)
6	逻辑运算符	&& !	一词一符	无
7	界限符	;() = { } [] ...	一词一符	无

➤ 语法分析器依据文法分析token串, 符号的集合即是文法的终结符集合。

例:语句if count>7 then result := 3.14;

```
if      <IF, - >
count  <ID, count>
>      <relop, - >
7      <INT, 7 >
then   <THEN, - >
result <ID, result>
:=     <ASSIGN, - >
3.14   <FLOAT, 3.14>
;      <SEMIC, - >
```

<一词一符, - >

<ID, 词素 >

<一类一符, 数值 >

使用正则文法即可定义每类单词构成的语言，是正则语言

3.1 单词的描述

➤ **正则表达式**(*Regular Expression, RE*) 是一种用来描述**正则语言**的更**紧凑**的表示方法。

➤ 例：正则语言 $L = \{a\}\{a, b\}^*(\{\varepsilon\} \cup (\{.,_ \}\{a, b\}\{a, b\}^*))$

正则表达式 $r = a(a|b)^*(\varepsilon | (.|_)(a|b)(a|b)^*)$

➤ 正则表达式可以由**较小的正则表达式**按照特定规则**递归**地构建。每个**正则表达式** r 定义（表示）一个**语言**，记为 $L(r)$ 。这个语言也是根据 r 的**子表达式**所表示的语言**递归定义**的

正则表达式的定义

- ε 是一个 RE , $L(\varepsilon) = \{\varepsilon\}$
- 如果 $a \in \Sigma$, 则 a 是一个 RE , $L(a) = \{a\}$
- 假设 r 和 s 都是 RE , 表示的语言分别是 $L(r)$ 和 $L(s)$, 则
 - $r|s$ 是一个 RE , $L(r|s) = L(r) \cup L(s)$
 - rs 是一个 RE , $L(rs) = L(r) L(s)$
 - r^* 是一个 RE , $L(r^*) = (L(r))^*$
 - (r) 是一个 RE , $L((r)) = L(r)$

运算的优先级: $*$ 、连接、 $|$

➤ 令 $\Sigma = \{a, b\}$, 则

$$\text{➤ } L(a|b) = L(a) \cup L(b) = \{a\} \cup \{b\} = \{a, b\}$$

$$\text{➤ } L((a|b)(a|b)) = L(a|b) L(a|b) = \{a, b\} \{a, b\} = \{aa, ab, ba, bb\}$$

$$\text{➤ } L(a^*) = (L(a))^* = \{a\}^* = \{\varepsilon, a, aa, aaa, \dots\}$$

$$\text{➤ } L((a|b)^*) = (L(a|b))^* = \{a, b\}^* = \{\varepsilon, \underline{a}, \underline{b}, \underline{aa}, \underline{ab}, \underline{ba}, \underline{bb}, \underline{aaa}, \dots\}$$

$$\text{➤ } L(a|a^*b) = L(a) \cup L(a^*b) = \{a, b, ab, aab, aaab, \dots\}$$

例：C语言无符号整数的 RE

➤ 十进制整数的 RE

➤ $(1|...|9)(0|...|9)^*|0$

➤ 八进制整数的 RE

➤ $0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$

➤ 十六进制整数的 RE

➤ $0x(0|1|...|9|a|...|f|A|...|F)(0|...|9|a|...|f|A|...|F)^*$

➤ 可以用 RE 定义的语言叫做

正则语言(*regular language*)或**正则集合**(*regular set*)

➤ 如果两个正则表达式 r 和 s 表示**同样的语言**，
则称 **r 和 s 等价**，记作 **$r=s$** 。

RE的代数定律

定律	描述
$r \mid s = s \mid r$	$\mid$ 是可以交换的
$r \mid (s \mid t) = (r \mid s) \mid t$	$\mid$ 是可结合的
$r (s t) = (r s) t$	连接是可结合的
$r (s \mid t) = r s \mid r t ;$ $(s \mid t) r = s r \mid t r$	连接对 $\mid$ 是可分配的
$\varepsilon r = r \varepsilon = r$	ε 是连接的单位元
$r^* = (r \mid \varepsilon)^*$	闭包中一定包含 ε
$r^{**} = r^*$	$*$ 具有幂等性

- 对任何正则文法 G ，存在定义同一语言的正则表达式 r
- 对任何正则表达式 r ，存在生成同一语言的正则文法 G

例: C标识符的右线性文法

- ① $S \rightarrow a \mid b \mid c \mid \dots \mid z \mid _$
- ② $S \rightarrow aT \mid bT \mid cT \mid dT \mid \dots \mid zT \mid _T$
- ③ $T \rightarrow a \mid b \mid c \mid d \mid \dots \mid z \mid 0 \mid 1 \mid 2 \mid 3 \mid \dots \mid 9$
- ④ $T \rightarrow aT \mid bT \mid cT \mid \dots \mid zT \mid _T \mid 0T \mid 1T \mid 2T \mid 3T \mid \dots \mid 9T$

➤ **正则定义**是具有如下形式的**定义序列**:

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_n \rightarrow r_n$$

给一些**RE命名**，并在之后的**RE**中像使用**字母表中的符号**一样使用这些**名字**

其中:

➤ 每个 d_i 都是一个**新符号**，它们都不在字母表 Σ 中，而且**各不相同**

➤ 每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的**正则表达式**

➤ C语言中标识符的正则定义

➤ $digit \rightarrow 0|1|2|\dots|9$

➤ $letter_ \rightarrow A|B|\dots|Z|a|b|\dots|z|_$

➤ $id \rightarrow letter_ (letter_|digit)^*$

➤ (整型或浮点型) 无符号数的正则定义

➤ $digit \rightarrow 0|1|2|\dots|9$

➤ $digits \rightarrow digit\ digit^*$

思考：假设整数部分不能以0开头，
如何修改？

➤ $optionalFraction \rightarrow .digits|\epsilon$

➤ $optionalExponent \rightarrow (E(+|-|\epsilon)digits)|\epsilon$

➤ $number \rightarrow digits\ optionalFraction\ optionalExponent$

2	2.15	2.15E+3	2.15E-3	2.15E3	2E-3
---	------	---------	---------	--------	------

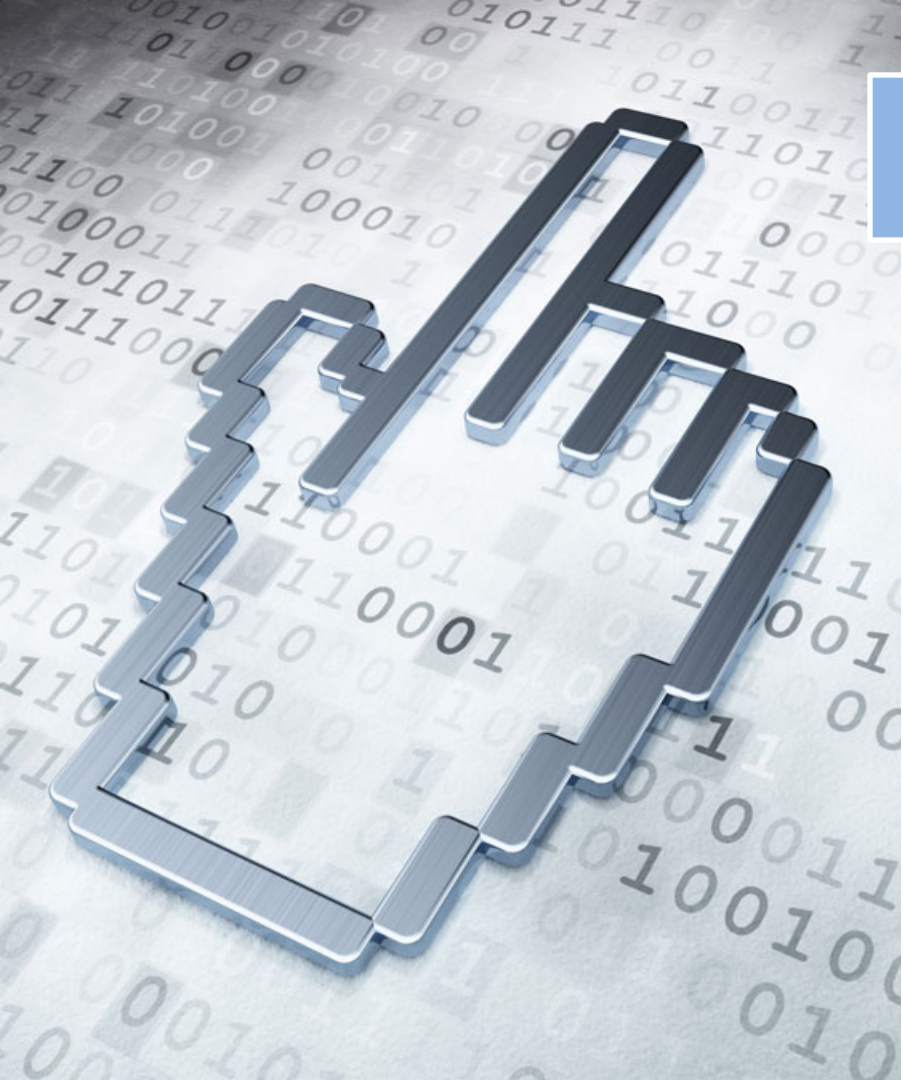
➤列出C--语言中合法的所有符号的正则定义

参考材料：QQ群文件/实验教程/附录_A_C--语言文法

7.1.1 Tokens

```
INT → /* A sequence of digits without spaces1 */  
FLOAT → /* A real number consisting of digits and one decimal point. The decimal  
point must be surrounded by at least one digit2 */  
ID → /* A character string consisting of 52 upper- or lower-case alphabetic, 10  
numeric and one underscore characters. Besides, an identifier must not start with a  
digit3 */  
SEMI → ;  
COMMA → ,  
ASSIGNOP → =  
RELOP → > | < | >= | <= | == | !=
```

2.2.4 Flex: 书写正则表达式



提纲

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

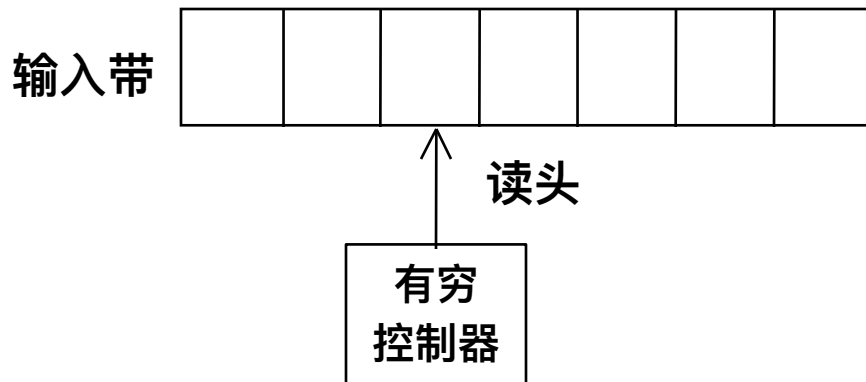
3.2 单词的识别

- 3.2.1 有穷自动机 (*Finite Automata*)
- 3.2.2 有穷自动机的分类
- 3.2.3 从正则表达式到有穷自动机
- 3.2.4 识别单词的 *DFA*

3.2.1 有穷自动机

- 有穷自动机 (*Finite Automata*, *FA*)由两位神经物理学家*McCulloch*和*Pitts*于1948年首先提出, 是对一类处理系统建立的数学模型
- 这类系统具有一系列离散的输入输出信息和有穷数目的内部状态
(状态: 概括了对过去输入信息处理的状况)
- 系统只需要根据当前所处的状态和当前面临的输入信息就可以决定系统的后继行为。每当系统处理了当前的输入后, 系统的内部状态也将发生改变

FA模型



- **输入带**(*input tape*): 用来存放输入符号串
- **读头**(*head*): 从左向右逐个读取输入符号, 不能修改 (只读)、不能往返移动
- **有穷控制器**(*finite control*): 具有有穷个状态数, 根据**当前的状态**和**当前输入符号**控制转入**下一状态**

FA的表示

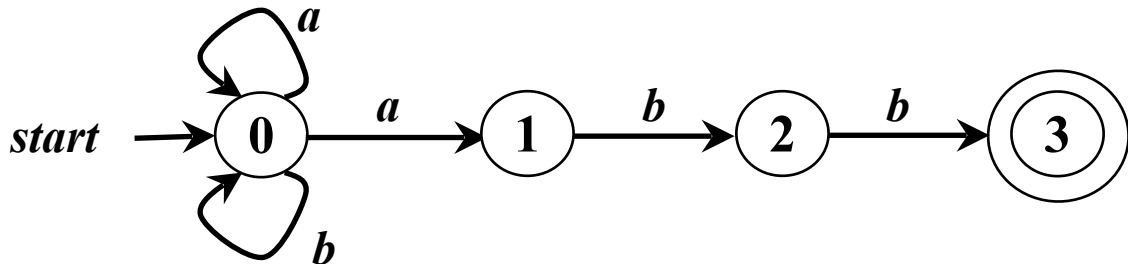
➤ 转换图 (Transition Graph)

➤ 结点：FA的状态

➤ 初始状态：只有一个，由 $start$ 箭头指向

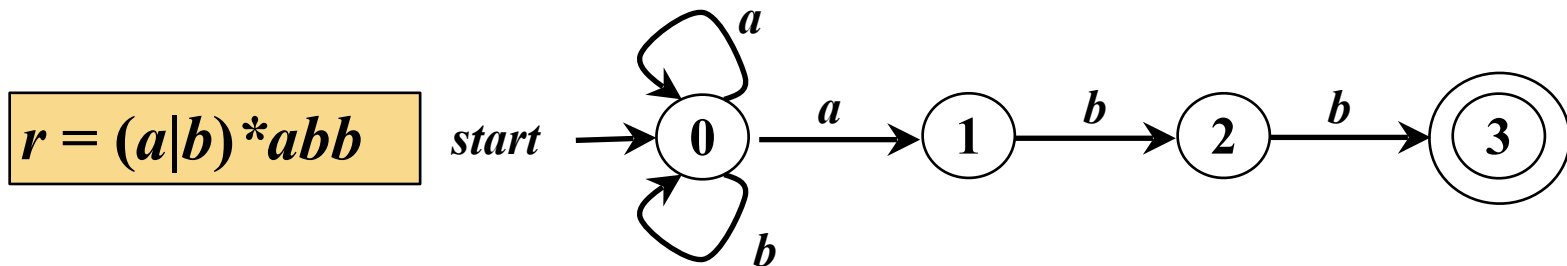
➤ 接收状态：可以有多个，用双圈表示

➤ 带标记的有向边（转换函数）：如果对于输入 a ，存在一个从状态 p 到状态 q 的转换，就在 p 、 q 之间画一条有向边，并标记上 a



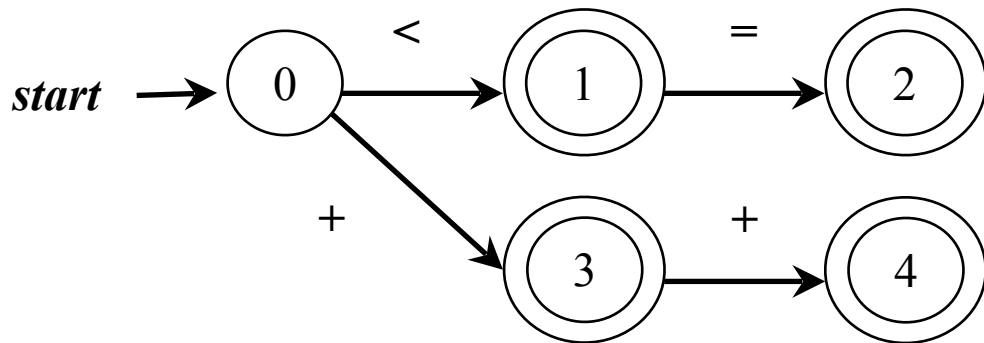
FA定义 (接收) 的语言

- 给定输入串 x ，如果存在一个对应于串 x 的从初始状态到某个接收状态的转换序列，则称串 x 被该FA接收
- 由一个有穷自动机 M 接收的所有串构成的集合称为该FA定义 (或接收) 的语言，记为 $L(M)$



$L(M)$ 是字母表 $\{a, b\}$ 上所有以 abb 结尾的串的集合

- 当输入串的**多个前缀**与一个或多个模式匹配时，总是选择**最长的前缀**进行匹配



- 在到达某个终态之后，只要输入带上还有符号，*DFA*就继续前进，以便寻找**尽可能长的匹配**

3.2.2 *FA*的分类

- 确定的 FA (*Deterministic finite automata, DFA*)
- 非确定的 FA (*Nondeterministic finite automata, NFA*)

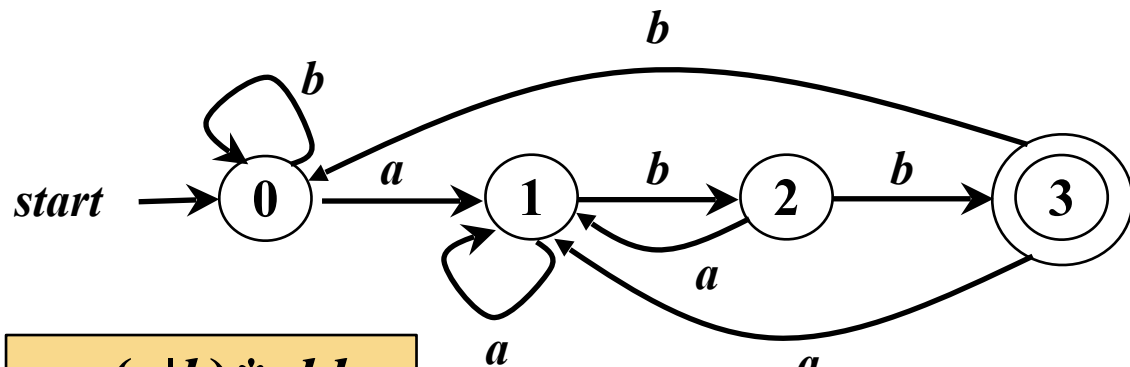
确定的有穷自动机 (DFA)

$$M = (S, \Sigma, \delta, s_0, F)$$

- S : 有穷状态集
- Σ : 输入字母表, 即输入符号集合。假设 ϵ 不是 Σ 中的元素
- δ : 将 $S \times \Sigma \rightarrow S$ 的转换函数。 $\forall s \in S, a \in \Sigma, \delta(s, a)$ 表示从状态 s 出发, 沿着标记为 a 的边所能到达的 (唯一) 状态
- s_0 : 开始状态 (或初始状态), $s_0 \in S$
- F : 接收状态 (或终止状态) 集合, $F \subseteq S$

例：一个 *DFA*

$$M = (S, \Sigma, \delta, s_0, F)$$



$$r = (a|b)^*abb$$

转换表

状态 \ 输入	输入	
	<i>a</i>	<i>b</i>
0	1	0
1	1	2
2	1	3
3	1	0

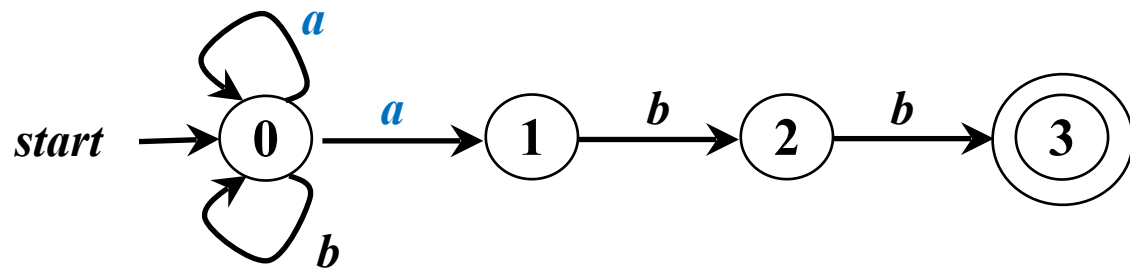
DFA 可以用 转换图 或 转换表 表示

$$M = (S, \Sigma, \delta, s_0, F)$$

- S : 有穷状态集
- Σ : 输入符号集合, 即输入字母表。假设 ε 不是 Σ 中的元素
- δ : 将 $S \times \Sigma \rightarrow 2^S$ 的转换函数。 $\forall s \in S, a \in \Sigma, \delta(s, a)$ 表示从状态 s 出发, 沿着标记为 a 的边所能到达的状态集合
- s_0 : 开始状态 (或初始状态), $s_0 \in S$
- F : 接收状态 (或终止状态) 集合, $F \subseteq S$

例：一个 NFA

$$M = (S, \Sigma, \delta, s_0, F)$$



$$r = (a|b)^*abb$$

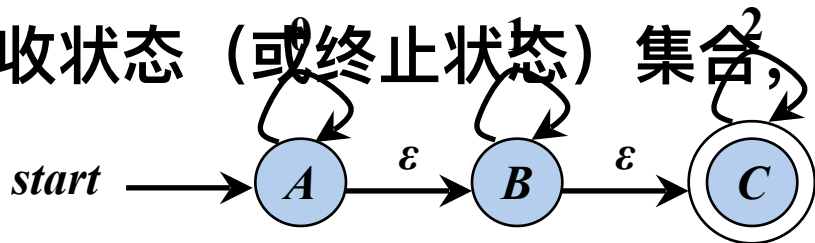
转换表

状态 \ 输入	a	b
0	$\{0, 1\}$	$\{0\}$
1	$\emptyset$	$\{2\}$
2	$\emptyset$	$\{3\}$
3.	$\emptyset$	$\emptyset$

如果转换函数没有给出对应于某个状态-输入对的信息，就把 $\emptyset$ 放入相应的表项中。

$$M = (S, \Sigma, \delta, s_0, F)$$

- S : 有穷状态集
- Σ : 输入符号集合, 即输入字母表。假设 ε 不是 Σ 中的元素
- δ : 将 $S \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^S$ 的转换函数。 $\forall s \in S, a \in \Sigma \cup \{\varepsilon\}$, $\delta(s, a)$ 表示从状态 s 出发, 沿着标记为 a 的边所能到达的状态集合
- s_0 : 开始状态 (或初始状态), $s_0 \in S$
- F : 接收状态 (或终止状态) 集合, $F \subseteq S$

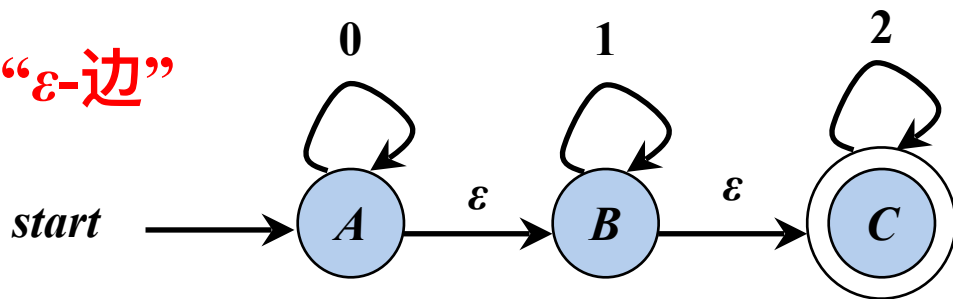


$$r = 0^*1^*2^*$$

带有和不带有“ ϵ -边”的NFA的等价性

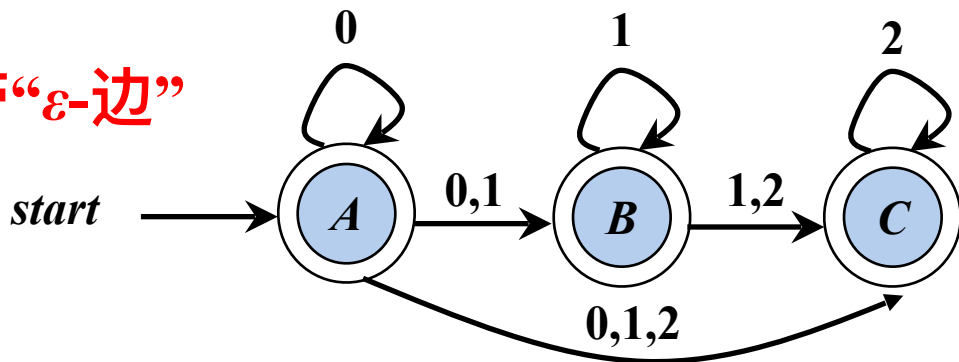
➤ 例

带“ ϵ -边”



$$r = 0^*1^*2^*$$

不带“ ϵ -边”

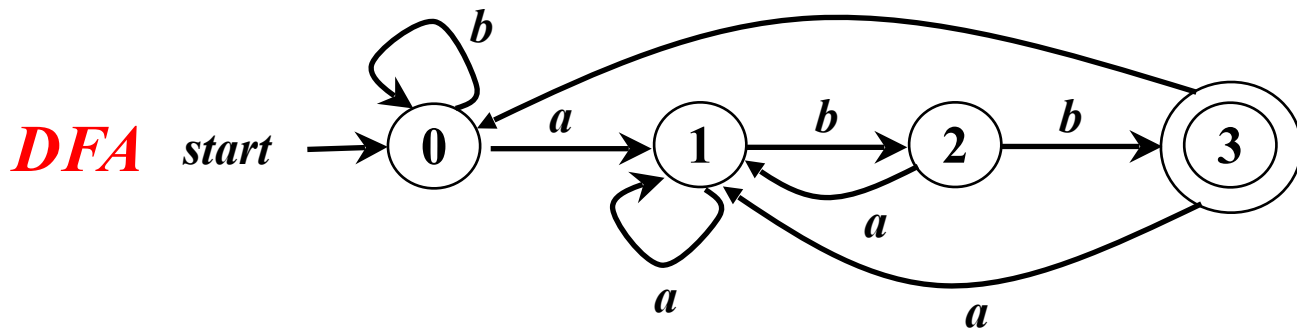
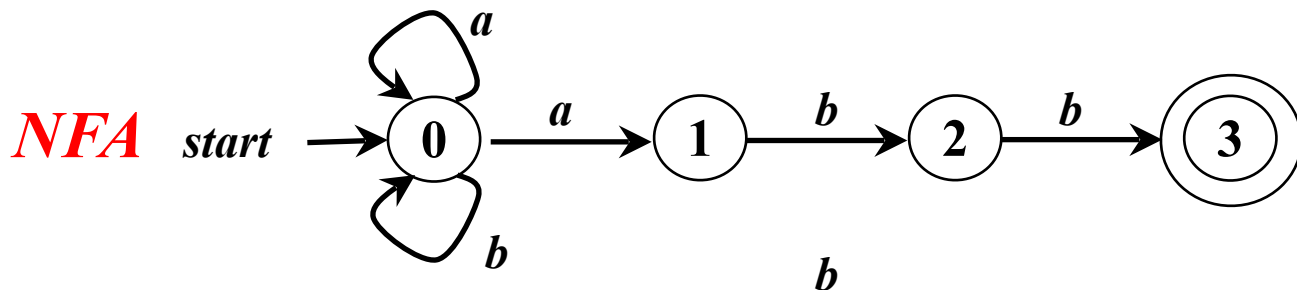


状态A: 0^*

状态B: 0^*1^*

状态C: $0^*1^*2^*$

➤ DFA和NFA可以识别相同的语言



状态1: 串以 a 结尾

状态2: 串以 ab 结尾

状态3: 串以 abb 结尾

$r = (a|b)^*abb$

正则文法 $\Leftrightarrow$ 正则表达式 $\Leftrightarrow$ FA

- 对任何NFA N ，存在定义同一语言的DFA D
 - 对任何DFA D ，存在定义同一语言的NFA N
- NFA在识别串时，从一个状态出发可能有多条标有相同符号的边，导致算法在实现时需要回溯，影响效率
 - DFA在识别串时，给定当前状态和输入符号，每一步都有确定的转移状态，识别效率高。

DFA的算法实现 (DFA模拟器)

- 输入：以文件结束符eof结尾的字符串 x 。DFA D 的开始状态 s_0 ，接收状态集 F ，转换函数 $move$ 。
- 输出：如果 D 接收 x ，则回答“yes”，否则回答“no”。
- 方法：将下述算法应用于输入串 x 。

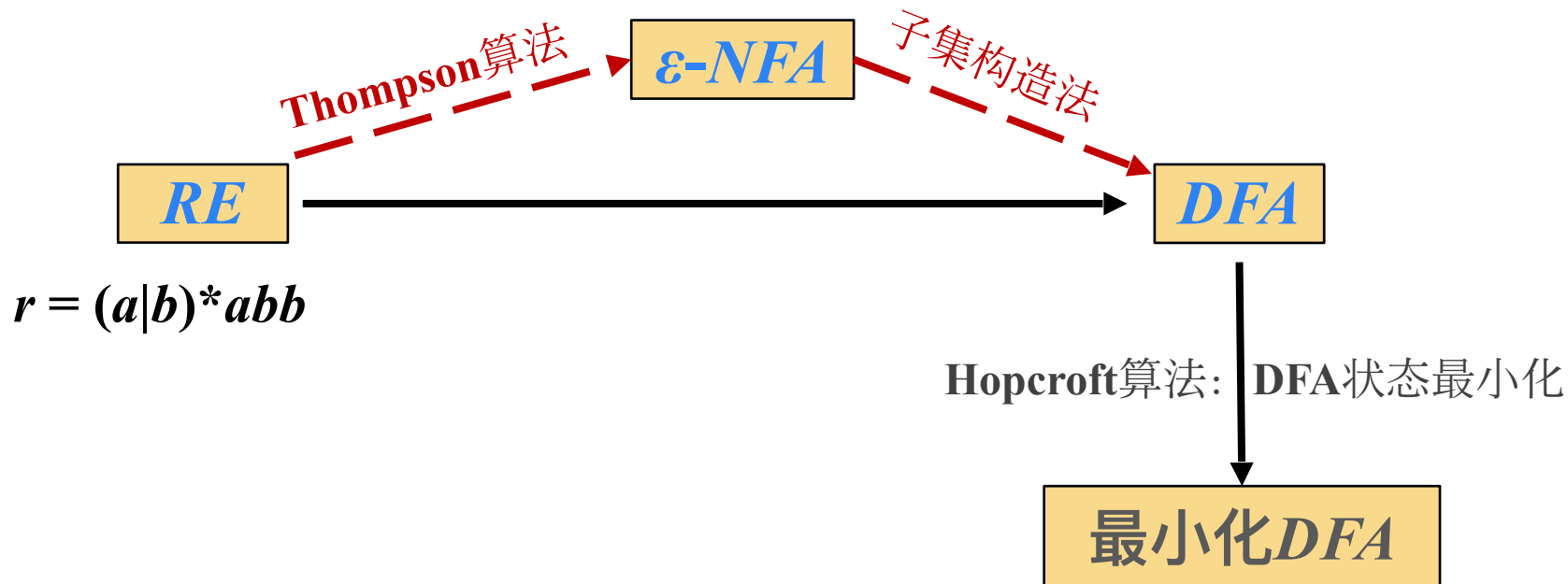
```
 $s = s_0$ ;  
 $c = nextChar()$ ;  
while ( $c \neq eof$ ) {  
     $s = move(s, c)$ ;  
     $c = nextChar()$ ;  
}  
if ( $s$ 在 $F$ 中) return “yes”;  
else return “no”;
```

➤ 函数 $nextChar()$ 返回输入串 x 的下一个符号

➤ 函数 $move(s, c)$ 表示从状态 s 出发，沿着标记为 c 的边所能到达的状态

思考：此模拟器代码是否依赖正则表达式的定义？ $move(s, c)$ 函数需访问DFA转换表？如何生成DFA转换表？

3.2.3 从正则表达式到有穷自动机



根据 RE 构造 ε - NFA (Thompson算法)

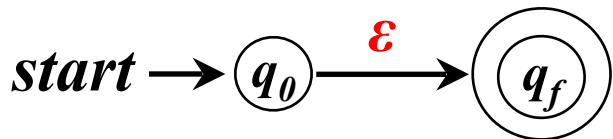
➤ 算法3.25 Thompson算法:

➤ 对于 ε 、符号 a 等基本符号, 直接构造其 ε - NFA

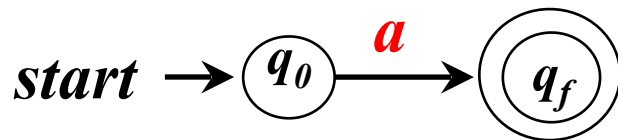
➤ 对于复合正则表达式 (连接、或和闭包), 使用归纳规则构造其 ε - NFA

基本规则: 基本符号的 ε - NFA

➤ 对于 ε , 有 ε - NFA



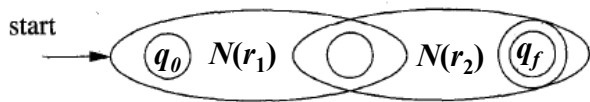
➤ $\forall a \in \Sigma$, 对于 a , 有 ε - NFA



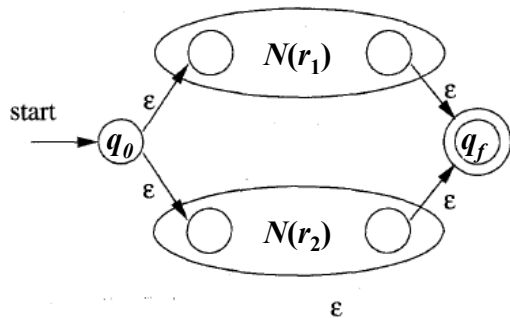
根据 RE 构造 ε - NFA (Thompson算法)

归纳规则：假设正则表达式 r_1 和 r_2 对应的 NFA 分别为 $N(r_1)$ 和 $N(r_2)$

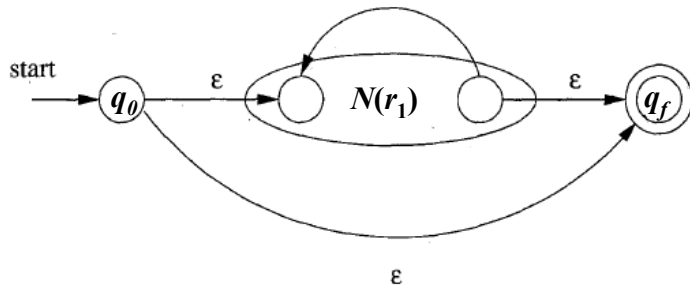
➤ $r = r_1 r_2$ 对应的 NFA



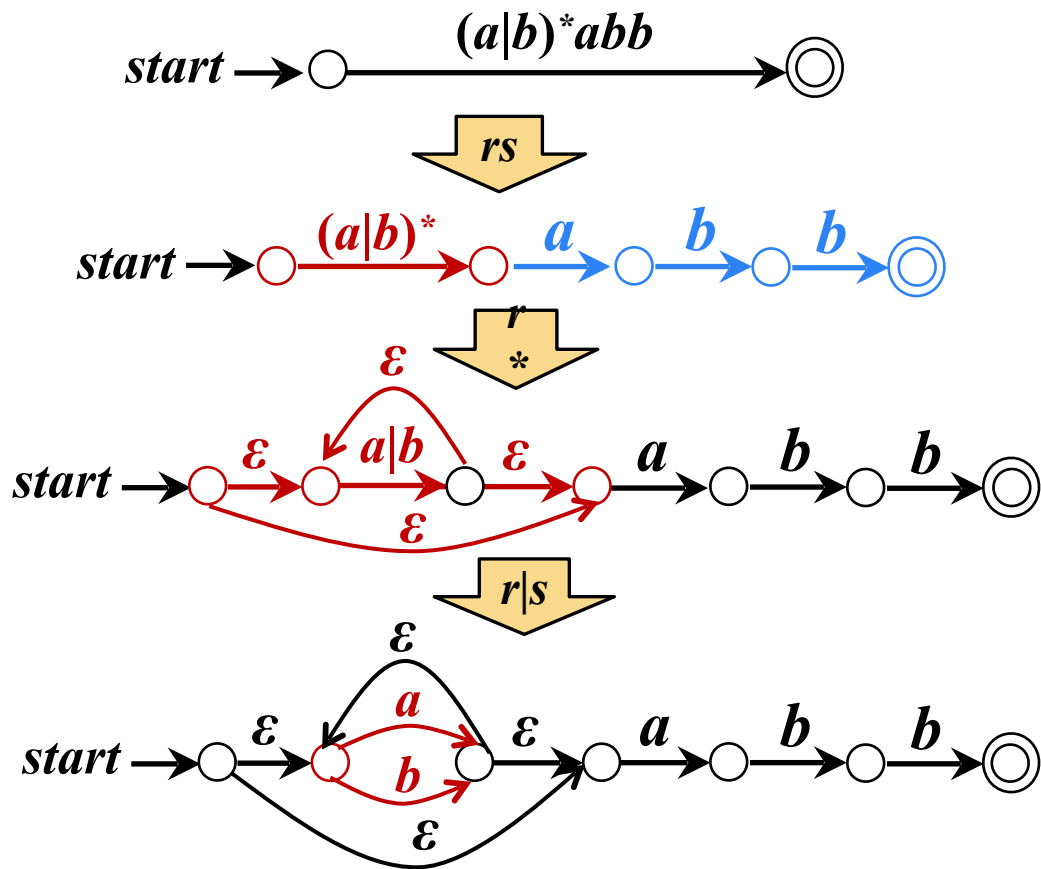
➤ $r = r_1 | r_2$ 对应的 NFA



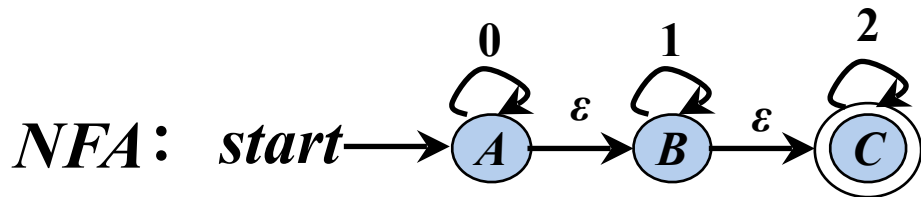
➤ $r = (r_1)^*$ 对应的 NFA



例: $r = (a|b)^*abb$ 对应的 *NFA*



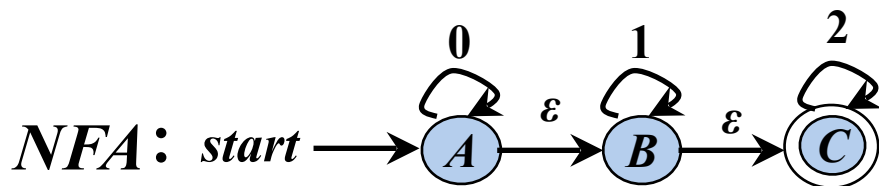
例：从带有 ε -NFA到DFA的转换



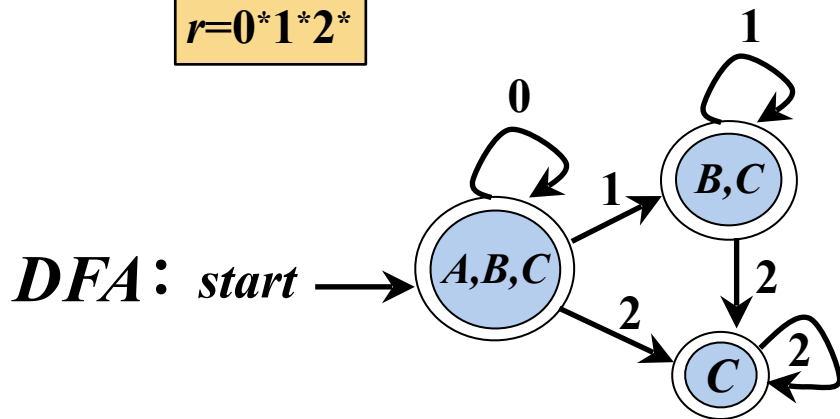
* 基础定义： ε -闭包

定义	表示	描述
状态 s 的 ε -闭包	$\varepsilon\text{-closure}(s)$	状态 s 以及从状态 s 开始经 ε 转换(边)可达的所有状态的集合
状态集 T 的 ε -闭包	$\varepsilon\text{-closure}(T)$	T 中的每个状态 $s \in T$ 的 ε -闭包的并集，即 $U_{s \in T} \varepsilon\text{-closure}(s)$

例：从带有 ε -NFA到DFA的转换



$$r = 0^*1^*2^*$$



$$move(T, a) = \bigcup_{s' \in T} \delta_{NFA}(s', a)$$

DFA转换表

NFA 状态集	DFA 状态	0	1	2
$\{A, B, C\} \cdot$	A'	$\{A, B, C\}$	$\{B, C\}$	$\{C\}$
$\{B, C\} \cdot$	B'	$\emptyset$	$\{B, C\}$	$\{C\}$
$\{C\} \cdot$	C'	$\emptyset$	$\emptyset$	$\{C\}$

- DFA的每个状态对应NFA的一个状态集。DFA开始状态：是NFA开始状态A的 ε -闭包
- $\delta_{DFA}(s, a) = \varepsilon\text{-closure}(\text{move}(T, a))$, 其中, T 为DFA状态 s 对应的NFA状态集
- 包含NFA接受状态的DFA状态是接受状态

子集构造法 (subset construction)

- 输入: $NFA\ N$
- 输出: 接收同样语言的 $DFA\ D$
- 方法: 一开始, $\varepsilon\text{-closure}(s_0)$ 是 $Dstates$ 中的唯一状态, 且它未加标记;
while (在 $Dstates$ 中有一个未标记状态 T) {
 给 T 加上标记;
 for (每个输入符号 a) {
 $U = \varepsilon\text{-closure}(\text{move}(T, a))$;
 if (U 不在 $Dstates$ 中)
 将 U 加入到 $Dstates$ 中, 且不加标记;
 $Dtran[T, a] = U$;
 }
}

操作	描述
$\varepsilon\text{-closure}(s)$	从 NFA 的状态 s 出发只通过 ε 转换能够到达的所有 NFA 状态的集合
$\varepsilon\text{-closure}(T)$	从 T 中的每个 NFA 状态 s 出发通过 ε 转换能够到达的所有 NFA 状态的集合, 即 $\bigcup_{s \in T} \varepsilon\text{-closure}(s)$
$\text{move}(T, a)$	从 T 中的每个状态 s 出发通过标号为 a 的转换能够到达的所有 NFA 状态的集合

计算 ε -closure (T)

将 T 的所有状态压入 $stack$ 中；

将 ε -closure (T)初始化为 T ；

while ($stack$ 非空) {

 将栈顶元素 t 给弹出栈中；

 for (每个满足如下条件的 u ：从 t 出发有一个标号为 ε 的转换到达状态 u)

 if (u 不在 ε -closure (T)中) {

 将 u 加入到 ε -closure (T)中；

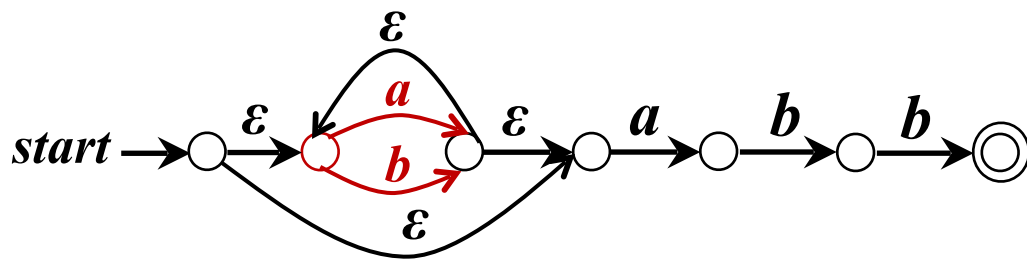
 将 u 压入栈中；

 }

 }

课后练习

- * 将NFA转换为DFA，画出转换表或转换图。



3.2.4 识别单词的 *DFA*

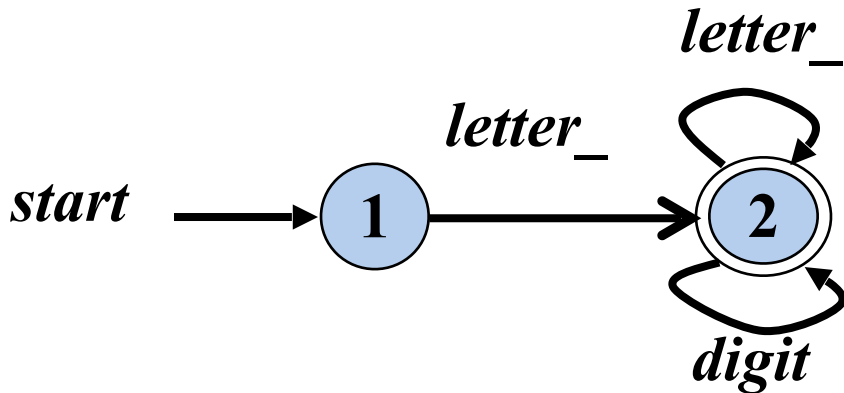
➤ 识别标识符的 *DFA*

标识符的词法规则：字母或下划线开头，字母数字、下划线组成的符号串

$digit \rightarrow 0|1|2|\dots|9$

$letter_ \rightarrow A|B|\dots|Z|a|b|\dots|z|_$

$id \rightarrow letter_ (letter_ | digit)^*$



识别无符号数的DFA

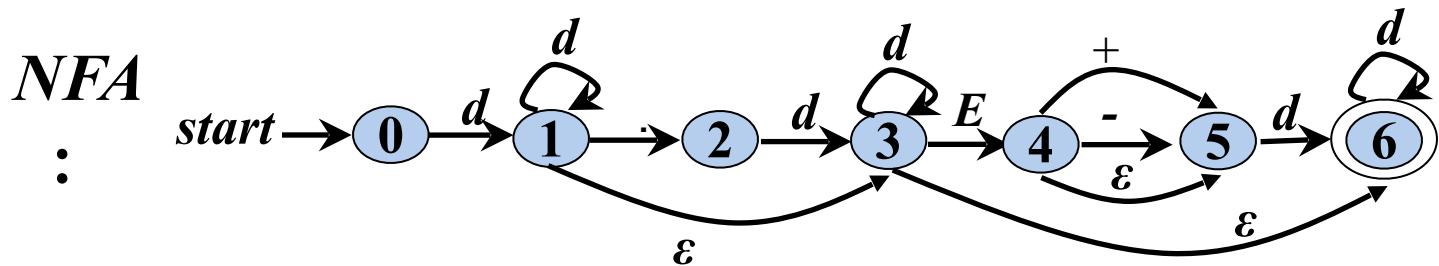
➤ $digit \rightarrow 0|1|2|\dots|9$

➤ $digits \rightarrow digit\ digit^*$

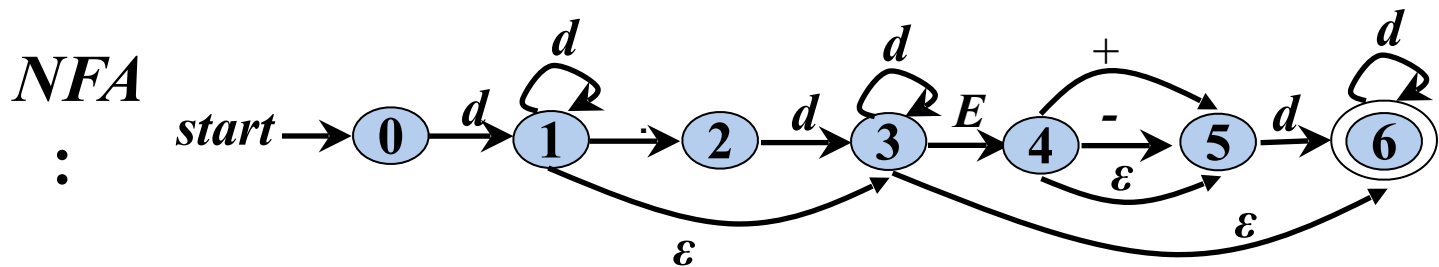
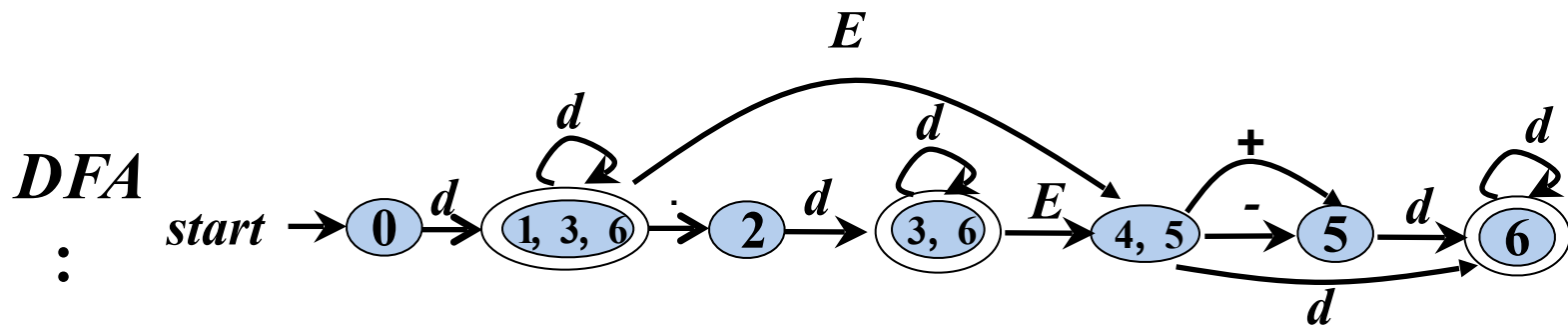
➤ $optionalFraction \rightarrow .digits|\epsilon$

➤ $optionalExponent \rightarrow (E(+|-|\epsilon)digits)|\epsilon$

➤ $number \rightarrow digits\ optionalFraction\ optionalExponent$

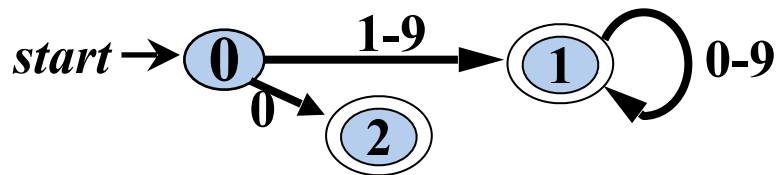


识别无符号数的 DFA

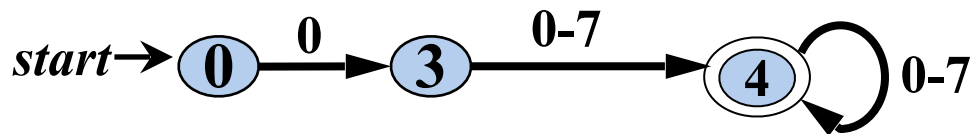


识别各进制无符号整数的DFA

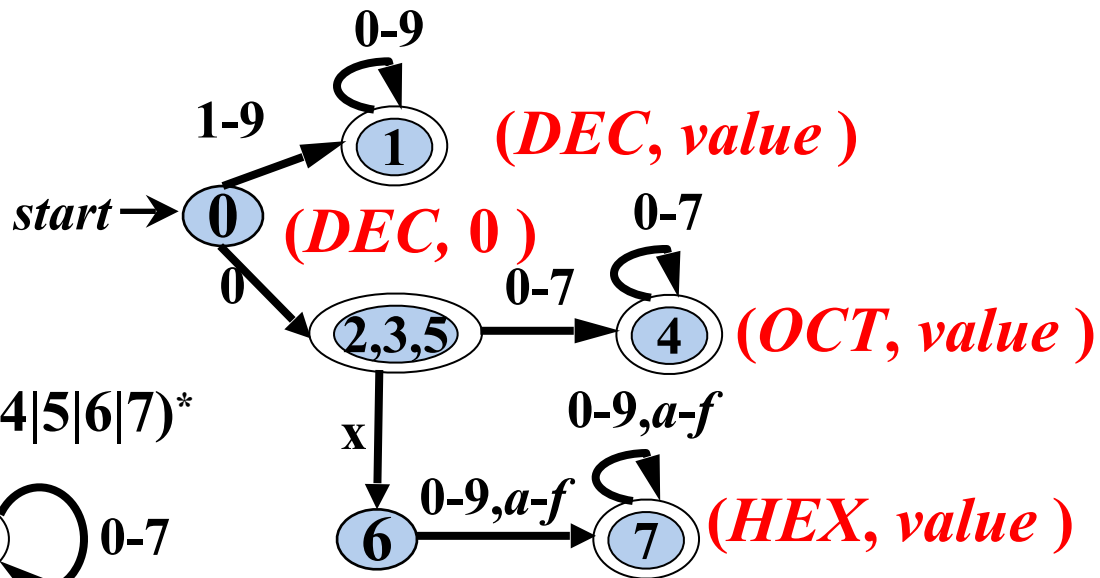
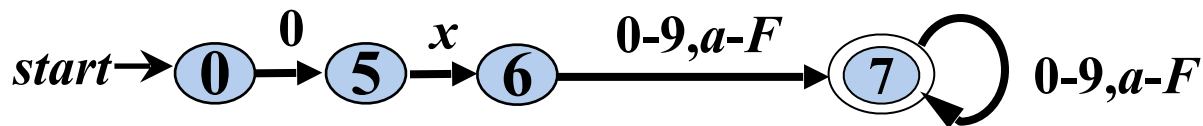
$DEC \rightarrow (1|...|9)(0|...|9)^* | 0$



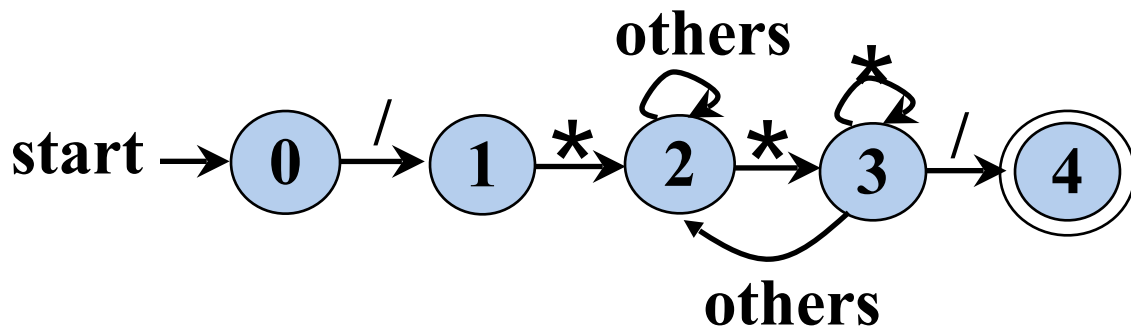
$OCT \rightarrow 0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$



$HEX \rightarrow 0x(0|1|...|9|a|...|f|A|...|F)(0|...|9|a|...|f|A|...|F)^*$

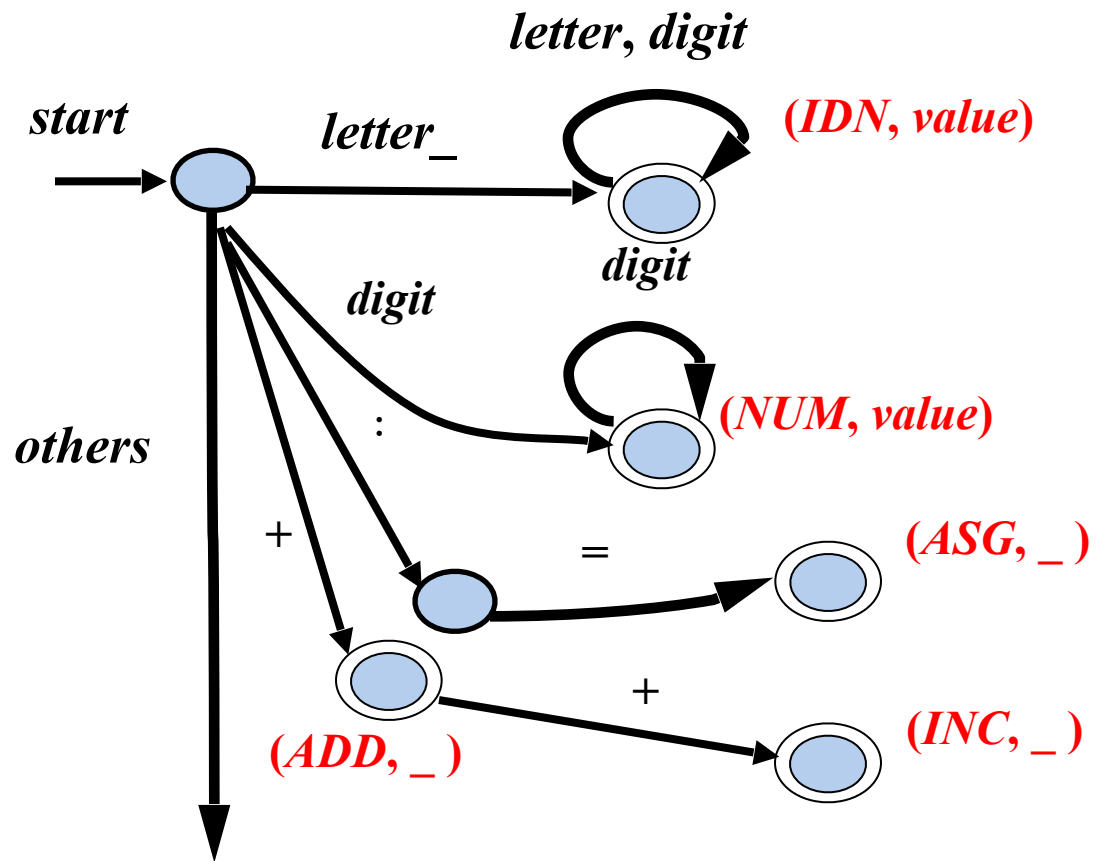


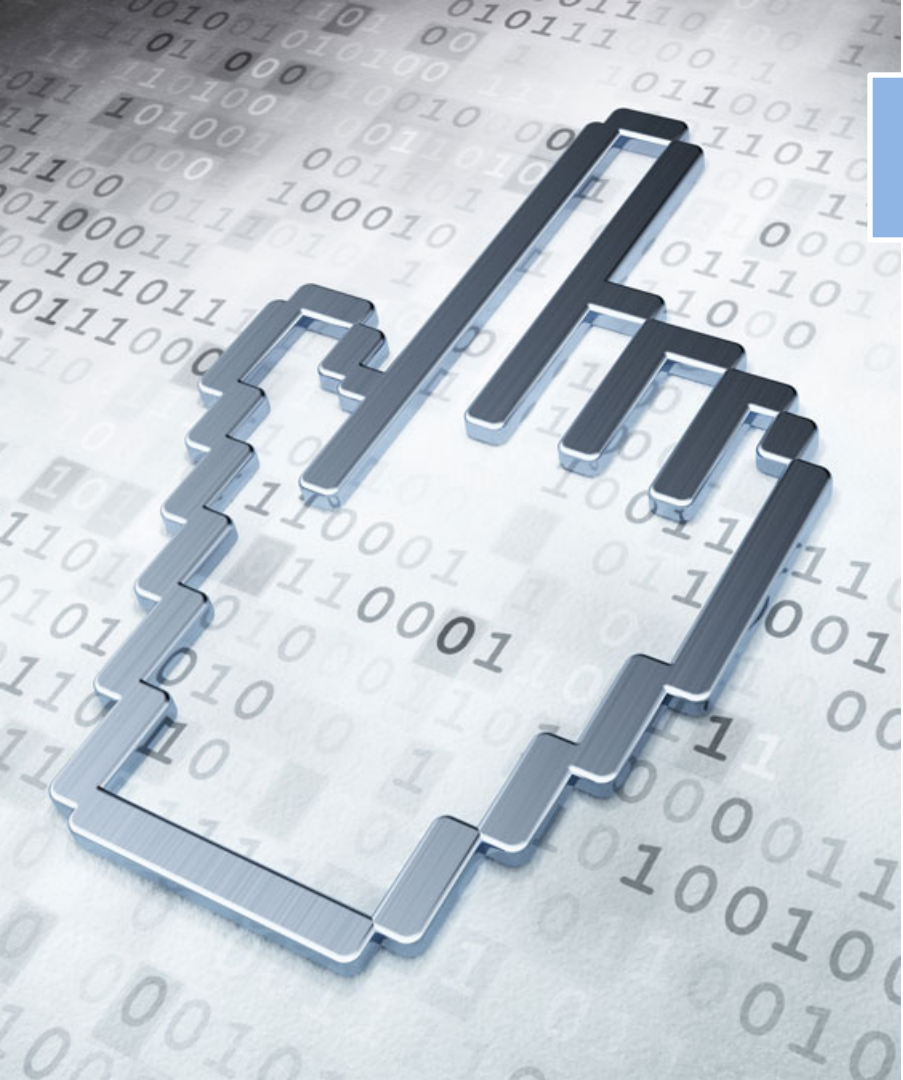
others:除了*和 / 之外的其它字符



思考: 如果注释文本中允许出现*和 /, 只要不出现“*/”子串, 如何定义?

识别 *Token* 的 *DFA*





提纲

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

3.4 词法分析阶段的错误处理

➤ 词法错误的类型

➤ 非法字符

➤ 例：~ @

➤ 单词拼写错误

➤ 例：*int i = 0x3G; float j = 1.05e; //G和e识别为标识符*

➤ 词法错误检测

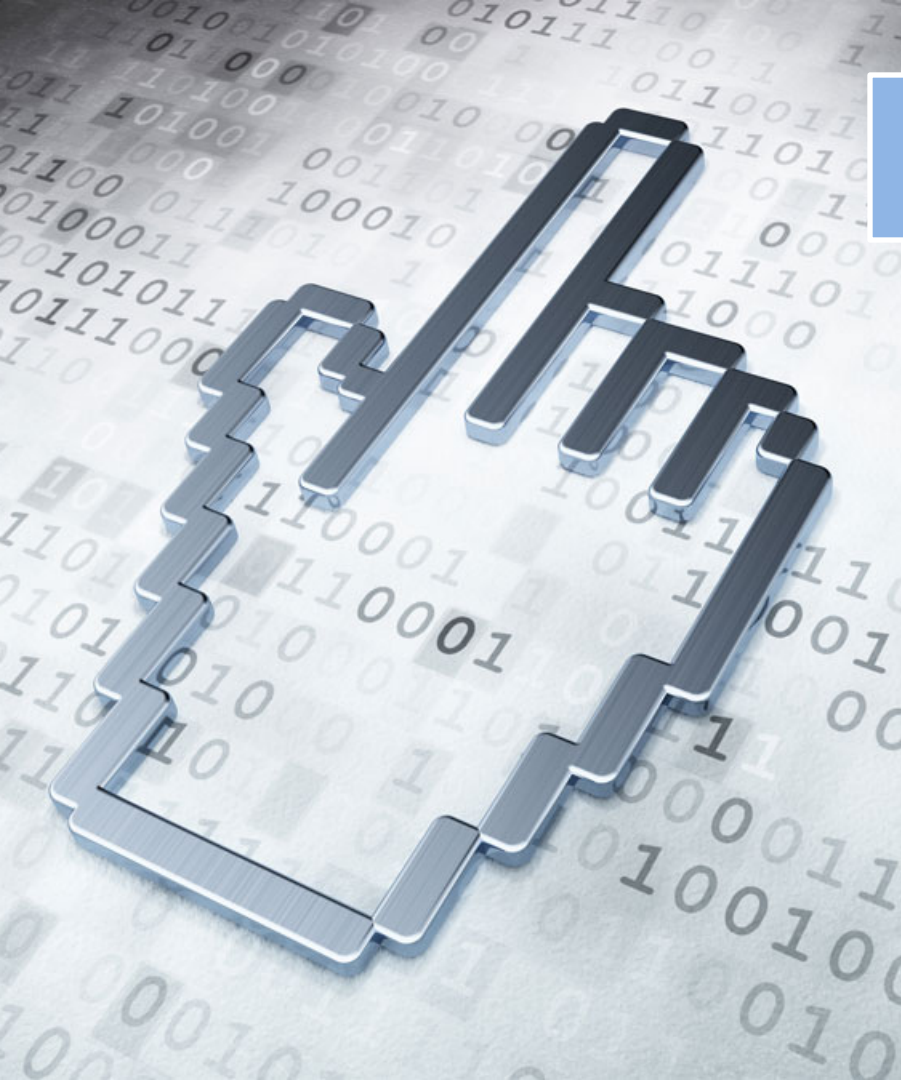
➤ 如果当前状态与当前输入符号在转换表对应项中的信息为空，而当前状态又不是终止状态，则调用错误处理程序

- 查找已扫描字符串中最后一个对应于某终态的字符
 - 如果**找到了**，将该字符与其前面的字符识别成一个单词。然后将输入指针退回到该字符，扫描器重新回到初始状态，继续识别下一个单词
 - 如果**没找到**，则确定出错，采用错误恢复策略

- 最简单的错误恢复策略：“**恐慌模式** (*panic mode*)”恢复
 - 从剩余的输入中不断删除字符，直到词法分析器能够在剩余输入的开头发现一个正确的字符为止
- 进行修补尝试
 - 删除一个多余的字符
 - 插入一个遗漏的字符
 - 用一个正确的字符代替一个不正确的字符；
 - 交换两个相邻的字符

1. 构建每类单词的正则表达式
2. 构建识别所有单词的DFA，并构建转换表
3. 开发DFA模拟器根据DFA转换表识别单词并输出
4. 加入错误处理

```
s = s0 ;  
c = nextChar ( ) ;  
while ( c != eof ) {  
    s = move ( s , c ) ; push(s);  
    if ( s == -1 ) rollback( );  
    c = nextChar ( ) ;  
}  
if (s在F中) return “yes”;  
else return “no”;
```



提纲

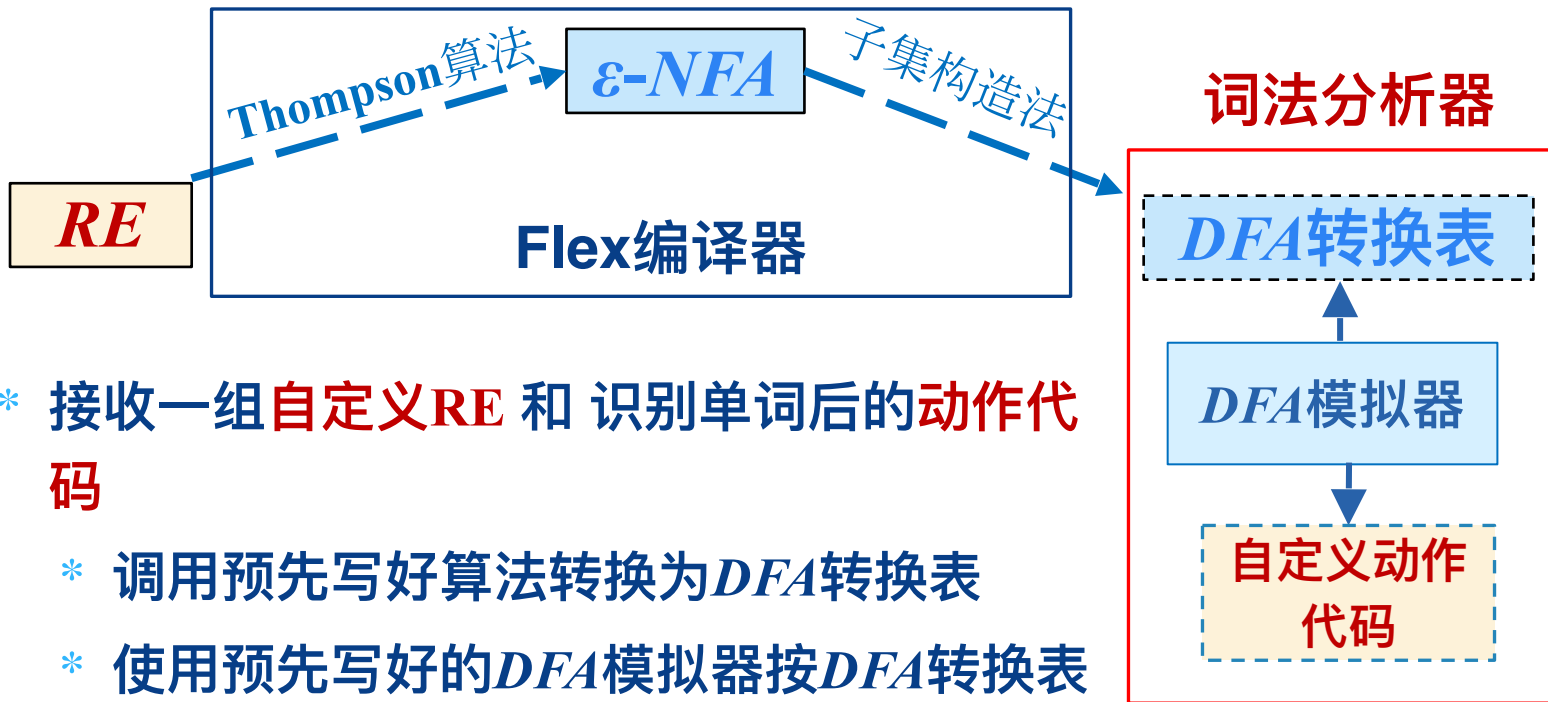
3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

自动生成词法分析器的基本思想



- * 接收一组**自定义RE**和识别单词后的**动作代码**
 - * 调用预先写好算法转换为**DFA转换表**
 - * 使用预先写好的**DFA模拟器**按**DFA转换表**识别各类单词，并根据单词类别调用自定义处理代码，即可生成词法分析器程序。

3.4 词法分析器生成工具Lex

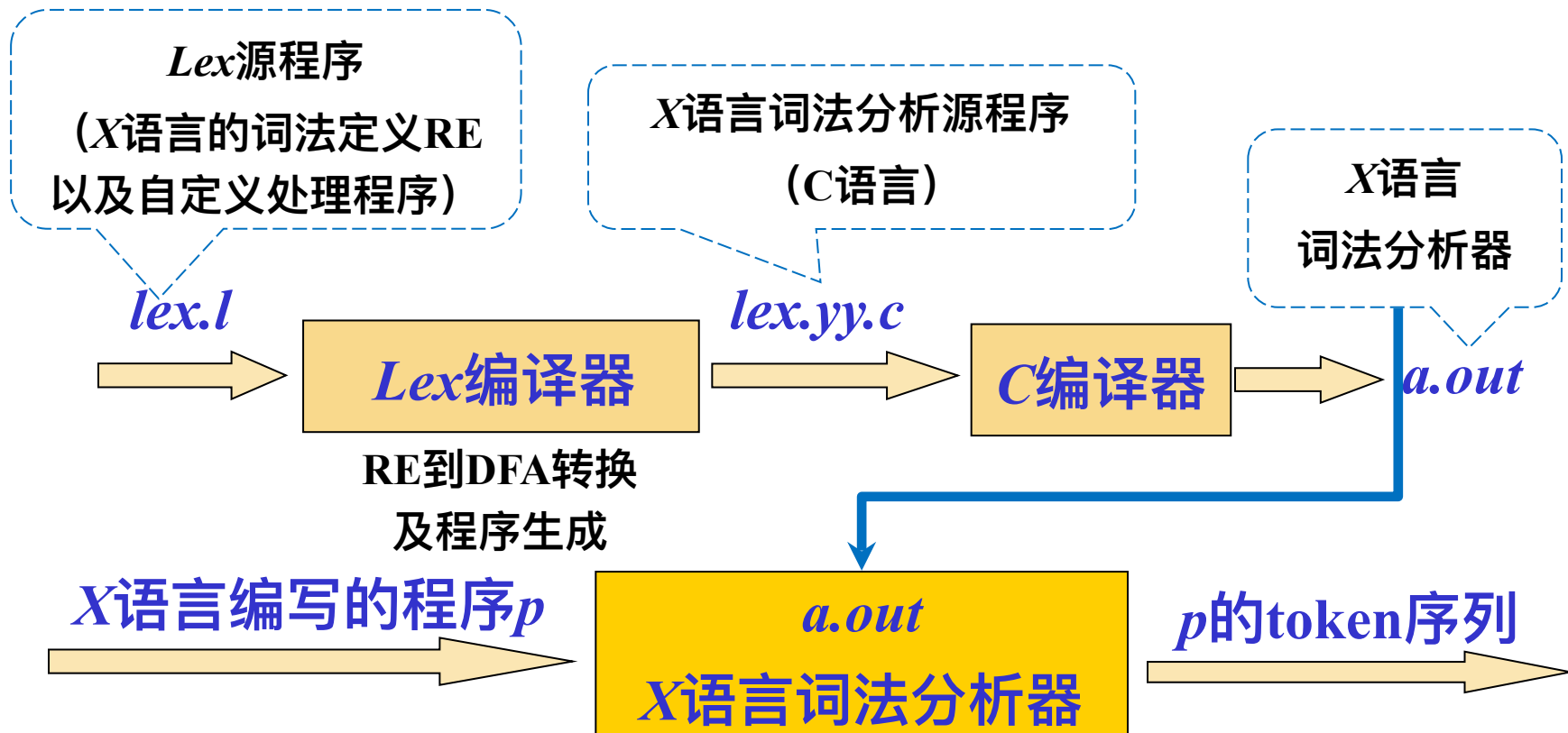
➤ Lex的构成

➤ Lex语言

➤ Lex编译器

Lesk, M.E. (October 1975). "Lex – A Lexical Analyzer Generator". *Comp. Sci. Tech. Rep. No. 39* (Murray Hill, New Jersey: Bell Laboratories).

Lex的使用



Flex 程序的结构

声明部分 (可选)

```
%{ /* 此处省略 #include 部分 */
```

```
int chars=0;  
int words = 0;  
int lines=0
```

所有内容都被直接复制到文件lex.yy.c中

```
%}
```

定义部分

```
letter [a-zA-Z]
```

正则定义

```
%%
```

```
{letter}+ {words++; chars+= yyleng; }
```

```
\n { chars++; linesars++ }
```

```
. { chars++; }
```

模式(RE) {动作}

转换规则

```
%%
```

```
int main(int argc, char** argv) {  
if (argc>1) { if (! (yyin = fopen(argv[1], "r")) ) {  
perror(argv[1]); return 1; } }
```

```
yylex();
```

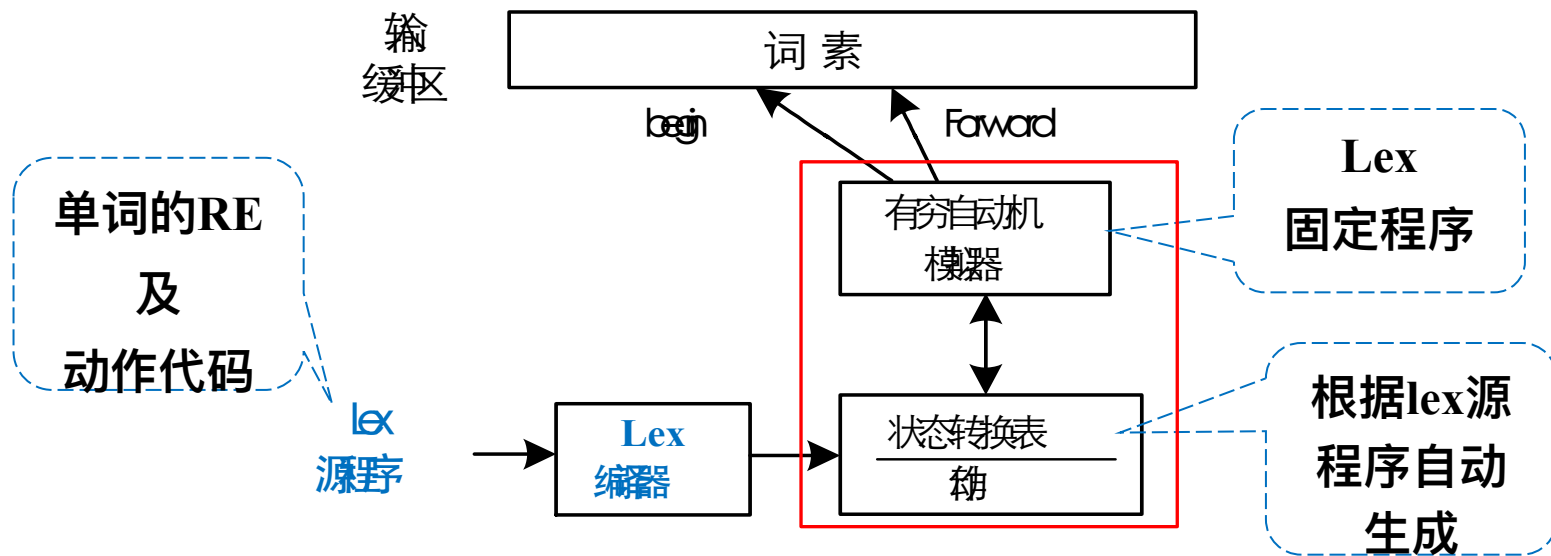
```
printf("%8d%8d%8d\n", words, chars); return 0; }
```

用户自定义代码

所有内容都被直接复制到文件lex.yy.c中

两组%%将代码分为三部分

Lex的功能是根据Lex源程序构造一个词法分析程序



Lex编译器根据Lex源程序中的RE生成DFA状态转换表



- * **Lex.yy.c**, 包含如下内容, 其核心是函数 **int yylex()**
 - * 基于RE词法生成的DFA转换表
 - * 固定的DFA模拟执行程序
 - * 从lex源代码中复制的C函数和动作代码
 - * 动作代码可以返回int类型的词法单元编码
 - * **yylval**默认是全局变量, 用于传递**词法单元的属性值**.
 - * 词法分析器中, 通过为**yylval**赋值而设置每个单词的属性值
 - * **Bison** 会自动将 **yylex()** 返回时的 **yylval** 赋值给对应符号的属性槽, 根据其在产生式右部的位置使用**\$1**、**\$2**或**\$3**等引用符号的属性值

- **Lex的构成加快了分析器的实现速度**
 - 程序员只需在很高的**模式层次**上描述软件，就可以依赖自动生成工具来生成详细的代码
- **修改扫描器的工作变得更加简单**
 - 只需修改那些受到影响的模式（RE），无需改写整个程序

实验一要求

- * 实验指导书-词法分析与语法分析
- * 利用Flex实现词法分析器
 - * 补全C--的词法规则
 - * 学习Flex使用，编写Flex源程序并编译
 - * 编写main函数调用词法分析函数yylex对输入c--源代码文件进行词法分析，并输出识别出的词法单元二元组和词法错误。（后面与语法分析集成）
 - * 对词法分析器进行测试
- * 实验教程-Project_1.pdf 必须通过2.1.6（必做内容）

< 实验教程-包含测试样例及输出

2026-03



实验教程-Project_3.pdf

829.5 KB



实验教程-Project_1.pdf

754.0 KB



实验教程_Project_2.pdf

732.2 KB

实验一要求

* 重要提示：

- * 三个实验任务是递进的，最终要构建一个完整的编译系统。
- * 注意系统地设计代码结构，将程序功能模块化，并优化模块关系和接口。
- * 功能模块要尽量松耦合，因为在某些实验中的输出或实现的任务（例如输出语法分析树），在后续实验中不再需要，要考虑模块易于剥离和维护。

- 单词的描述
 - 正则表达式
 - 正则定义
- 单词的识别
 - 有穷自动机
 - 有穷自动机的分类
 - 从正则表达式到有穷自动机
 - 识别单词的DFA
- 词法分析阶段的错误处理
- 词法分析器生成工具Lex

1. 绪论 (2学时)
2. 语言及其文法 (2学时)
3. 词法分析 (3学时)
4. 语法分析 (9学时)
5. 语法制导翻译 (6学时)
6. 中间代码生成 (7学时)
7. 运行时的存贮组织 (3学时)
8. 代码优化 (6学时)
9. 代码生成 (2学时)



结束

